<u>Machine Learning & Deep Learning</u>

Experiment 9

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

1. Dataset Source

- **Dataset Name:** IMDB Dataset of 50K Movie Reviews
- **Source:** [Kaggle - IMDB Dataset of 50K Movie Reviews]

2. Dataset Description

This dataset contains 50,000 highly polarized movie reviews from the Internet Movie Database (IMDB). It is a classic dataset used for binary sentiment classification.

- **Size:** 50,000 records, 2 columns.
- **Target Variable:** sentiment (Binary: "positive" or "negative").
- **Features:** * review: The raw text of the movie review.

3. Mathematical Formulation of the Algorithm

A. Standard Recurrent Neural Network (RNN):
Unlike standard feedforward networks, RNNs have loops that allow information to persist. The hidden state h_t at time t is a function of the input at time x_t and the hidden state from the previous time step h_(t-1).

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

B. Long Short-Term Memory (LSTM):
Standard RNNs suffer from the **Vanishing Gradient Problem**, making them unable to learn long-term dependencies in long sentences. LSTMs solve this by introducing a **Cell State** (C_t) and three gates (Forget, Input, Output) to regulate the flow of information.

1. **Forget Gate (f_t):** Decides what information to throw away from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input Gate (i_t) and Candidate Cell State (Ĉ_t):** Decides what new information to store.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

3. **Update Cell State (C_t):**

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. **Output Gate (o_t) and Hidden State (h_t):** Decides what to output based on the filtered cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

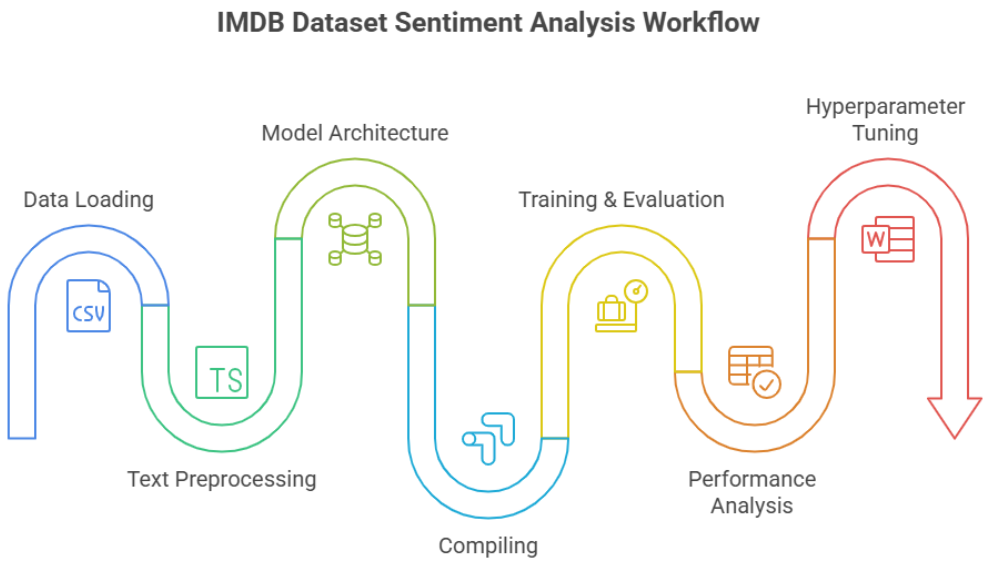
$$h_t = o_t * \tanh(C_t)$$

4. Algorithm Limitations

- **Computationally Expensive:** The complex gated architecture of LSTMs requires significantly more memory and training time compared to standard ANNs or CNNs.
- **Sequential Processing:** Because time step t relies on time step t-1, LSTMs cannot process data in parallel (unlike modern Transformer architectures).
- **Overfitting on Small Data:** LSTMs are highly prone to overfitting if the dataset is small; they require large amounts of text to generalize well.

5. Methodology / Workflow

1. **Data Loading:** Load IMDB_Dataset.csv.
2. **Text Preprocessing:** * Encode the labels (positive → 1, negative → 0).
    - **Tokenization:** Convert raw text words into integer sequences using a Vocabulary dictionary.
    - **Padding:** Neural networks require fixed-length inputs. We pad short reviews with zeros and truncate long reviews to a fixed maxlen (e.g., 200 words).
3. **Model Architecture:**
    - **Embedding Layer:** Converts integer word indices into dense, meaningful vectors of fixed size.
    - **LSTM Layer:** Processes the sequence of word embeddings.
    - **Dense Layer:** A final neuron with a Sigmoid activation to output a probability between 0 and
4. **Compiling:** Optimizer = 'Adam', Loss = 'Binary Crossentropy'.
5. **Training & Evaluation:** Fit the model and evaluate accuracy.



6. Performance Analysis

- **Metric:** Accuracy, Validation Loss.
- **Analysis:** Text data is inherently noisy. A well-trained LSTM on the IMDB dataset typically achieves an accuracy of **85% to 88%**. If the training accuracy approaches 99% but validation accuracy plateaus around 80%, the model has memorized the training vocabulary (overfitting).

7. Hyperparameter Tuning

- **Vocabulary Size (max_features):** Limiting the dictionary to the top 5,000 or 10,000 most frequent words removes rare words (noise).
- **Embedding Dimension:** The size of the vector representing each word (e.g., 64 or 128). Larger dimensions capture more semantic meaning but increase computational load.
- **LSTM Units:** The dimensionality of the hidden state (e.g., 64 or 128 units).
- **Dropout:** Adding Dropout and Recurrent Dropout inside the LSTM layer is crucial to force the network to not rely on specific word sequences, aiding generalization.

Code:

```
#
═══════════════════════════════════════
═══════════════
# EXPERIMENT 9: LSTM FOR TEXT
CLASSIFICATION (SENTIMENT)
#
═══════════════════════════════════════
═══════════════

import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text
import Tokenizer
from
tensorflow.keras.preprocessing.sequence
import pad_sequences
from tensorflow.keras.models import
Sequential
```

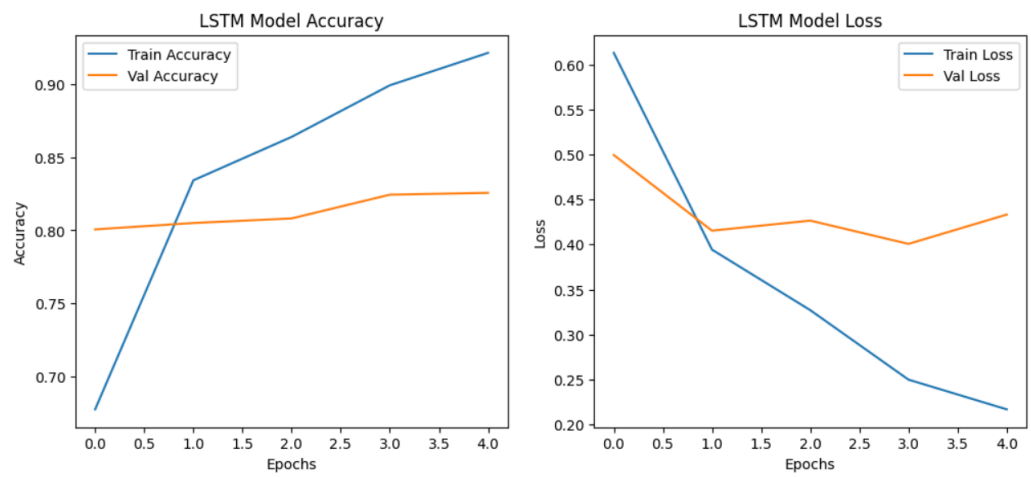
```python
from tensorflow.keras.layers import
Embedding, LSTM, Dense, Dropout
from sklearn.model_selection import
train_test_split
import matplotlib.pyplot as plt

print("--- EXPERIMENT 9: LSTM ON TEXT
DATA (IMDB REVIEWS) ---")

# 1. Load Dataset
try:
    # Sampling 10,000 rows for faster
training in this lab demonstration.
    # Remove .sample(10000) to use all
50,000 reviews.
    df =
pd.read_csv('IMDB_Dataset.csv').sample(1
0000, random_state=42)
    print("Dataset loaded
successfully.")
except FileNotFoundError:
    print("Error: 'IMDB_Dataset.csv' not
found. Please upload the file.")
    df = pd.DataFrame()

if not df.empty:
    # 2. Preprocessing
    # Encode labels
    df['sentiment'] =
df['sentiment'].map({'positive': 1,
'negative': 0})

    texts = df['review'].values
    labels = df['sentiment'].values

    # 3. Split Data
    X_train_text, X_test_text, y_train,
y_test = train_test_split(texts, labels,
test_size=0.2, random_state=42)

    # 4. Tokenization & Padding
    max_features = 5000  # Only consider
the top 5000 words in the dataset
    maxlen = 200         # Cut reviews
after 200 words

    print("Tokenizing text...")
    tokenizer =
Tokenizer(num_words=max_features)
    tokenizer.fit_on_texts(X_train_text)

    # Convert text to sequence of
integers
    X_train_seq =
tokenizer.texts_to_sequences(X_train_tex
t)
    X_test_seq =
tokenizer.texts_to_sequences(X_test_text
)

    # Pad sequences to ensure uniform
length
    X_train = pad_sequences(X_train_seq,
maxlen=maxlen)
    X_test = pad_sequences(X_test_seq,
maxlen=maxlen)

    print(f"Shape of X_train:
{X_train.shape}")
    print(f"Shape of X_test:
{X_test.shape}")

    #
==========================================
==
    # Build the LSTM Model
    #
==========================================
==
    embedding_dim = 64
```

```
    lstm_units = 64

    model = Sequential()

    # 1. Embedding Layer: Maps word
indices to dense vectors

model.add(Embedding(input_dim=max_featur
es, output_dim=embedding_dim,
input_length=maxlen))

    # 2. LSTM Layer
    model.add(LSTM(units=lstm_units,
dropout=0.2, recurrent_dropout=0.2))

    # 3. Output Layer: Sigmoid for
binary classification
    model.add(Dense(1,
activation='sigmoid'))

    model.summary()

    #
═══════════════════════════════════════
══
    # Compile and Train
    #
═══════════════════════════════════════
══
    model.compile(optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])

    print("\nTraining LSTM (this will
take a few minutes depending on
hardware)...")
    history = model.fit(X_train,
y_train,
                        batch_size=64,
                        epochs=5,
validation_split=0.2,
                        verbose=1)

    #
═══════════════════════════════════════
══
    # Evaluation
    #
═══════════════════════════════════════
══
    test_loss, test_acc =
model.evaluate(X_test, y_test,
verbose=0)
    print(f"\nFinal Test Accuracy:
{test_acc:.4f}")

    # Visualization: Accuracy & Loss
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)

plt.plot(history.history['accuracy'],
label='Train Accuracy')

plt.plot(history.history['val_accuracy']
, label='Val Accuracy')
    plt.title('LSTM Model Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'],
label='Train Loss')

plt.plot(history.history['val_loss'],
label='Val Loss')
    plt.title('LSTM Model Loss')
    plt.xlabel('Epochs')
```

```
    plt.ylabel('Loss')
    plt.legend()

    plt.show()

    # Optional: Test a custom review
    custom_review = ["This movie was absolutely fantastic! The acting was great and the plot was thrilling."]
    custom_seq = tokenizer.texts_to_sequences(custom_review)
    custom_padded = pad_sequences(custom_seq, maxlen=maxlen)
    prediction = model.predict(custom_padded)[0][0]

    print(f"\nCustom Review: '{custom_review[0]}'")
    print(f"Predicted Sentiment Probability: {prediction:.4f}")
    print("Result:", "Positive" if prediction > 0.5 else "Negative")
```

Output:

```
Training LSTM (this will take a few minutes depending on hardware)...
Epoch 1/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 29s 254ms/step - accuracy: 0.5947 - loss: 0.6634 - val_accuracy: 0.8006 - val_loss: 0.4994
Epoch 2/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 25s 250ms/step - accuracy: 0.8300 - loss: 0.4191 - val_accuracy: 0.8050 - val_loss: 0.4153
Epoch 3/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 24s 243ms/step - accuracy: 0.8765 - loss: 0.3106 - val_accuracy: 0.8081 - val_loss: 0.4265
Epoch 4/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 23s 231ms/step - accuracy: 0.9013 - loss: 0.2497 - val_accuracy: 0.8244 - val_loss: 0.4007
Epoch 5/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 24s 245ms/step - accuracy: 0.9253 - loss: 0.2153 - val_accuracy: 0.8256 - val_loss: 0.4332

Final Test Accuracy: 0.8430
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 553ms/step

Custom Review: 'This movie was absolutely fantastic! The acting was great and the plot was thrilling.'
Predicted Sentiment Probability: 0.9037
Result: Positive
```

Conclusion: In this experiment, we utilized an LSTM network to perform sentiment analysis on movie reviews. **Result:** By tuning hyperparameters like vocabulary size and applying dropout, the model demonstrated its capability to interpret human language and successfully classify text sentiment, despite the complexities of natural language phrasing.